# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY**
**COURSE CODE: CSA5117**

---

**COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage : 20%

**QUESTIONS:** **Total Marks: 50**

**Annexure A**

## Scenario-Based

---

***Code of Conduct:***

*I Namburi Sai Yashaswini(192425250)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: yashaswini***

## Annexure A
## Scenario-based

**1. Secure Data Transmission using Block Cipher**
A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.
B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

**2. Choosing an Encryption Algorithm for Banking System**
A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.
B. Justify your choice based on security strength and efficiency.

**3. Key Exchange in an Unsecured Network**
Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.
B. Identify a possible security threat and suggest a solution to mitigate it.

**4. Public Key Cryptography for Email Security**
An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.
B. Explain how key distribution and management play a role in maintaining security.

**5. Cryptography for IoT Devices**
A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.
B. Recommend a suitable cryptographic approach and justify your answer.

# Scenario 2: Choosing an Encryption Algorithm for Banking System

> **Scenario:** A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES and Blowfish. (a) Identify the most suitable algorithm. (b) Justify the choice based on security strength and efficiency.

## 1. Understanding the Scenario

Banking systems process card payments, fund transfers and account updates that must stay confidential and unaltered while moving between customers, application servers, databases and payment networks. An algorithm for this setting is judged on the following requirements:

- **Strong confidentiality:** a key space large enough that brute-force search is impractical for the lifetime of the data.
- **Moderate performance:** transactions are short messages, so throughput matters less than correctness, but latency must stay acceptable at high volume.
- **Standards and compliance:** financial regulators and auditors expect algorithms that are standardised and widely reviewed.
- **Interoperability:** ATMs, HSMs, EMV cards and inter-bank networks already use a fixed set of ciphers.

## 2. Overview of the Candidate Algorithms

### 2.1 DES (Data Encryption Standard)

DES is a 16-round Feistel cipher that encrypts 64-bit blocks using a 56-bit effective key (64 bits including 8 parity bits). It was standardised in 1977 and was the dominant cipher for decades. Its 56-bit key space (2^56 keys) is now small enough to search exhaustively; the EFF's DES Cracker recovered a key in under three days in 1998, and modern hardware or cloud clusters do it far faster. DES is therefore unsuitable for protecting financial data.

### 2.2 3-DES (Triple DES)

3-DES strengthens DES by applying it three times to every block in an Encrypt-Decrypt-Encrypt (EDE) sequence, using two or three independent keys. The EDE form also keeps backward compatibility, because setting K1 = K2 = K3 reduces it to single DES.

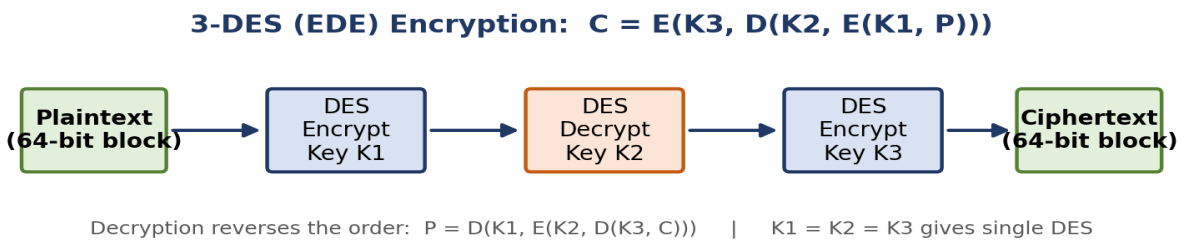



*Figure 1: 3-DES (EDE) encryption and decryption structure*

Using three keys gives a 168-bit key length, but the meet-in-the-middle attack reduces the effective strength to about 112 bits; with two keys the effective strength is about 80 bits under NIST's analysis. Either is far above DES. Each block passes through 48 rounds, which makes 3-DES roughly three times slower than DES, a cost that is acceptable under the scenario's moderate-performance requirement.

### 2.3 Blowfish

Blowfish, designed by Bruce Schneier in 1993, is a 16-round Feistel cipher with a 64-bit block and a variable key length from 32 to 448 bits. It uses key-dependent S-boxes and an 18-entry P-array, which makes encryption very fast in software once the keys are set up. However, key setup is expensive (about 4 KB of subkeys are generated through 521 encryption iterations), it is not an official standard, and its 64-bit block has the same birthday-bound weakness as 3-DES. Its own designer recommends Twofish for new systems.

## 3. Comparative Analysis

**Table 1: Comparison of DES, 3-DES and Blowfish**

| Parameter | DES | 3-DES | Blowfish |
|---|---|---|---|
| **Block size** | 64 bits | 64 bits | 64 bits |
| **Key size** | 56 bits (effective) | 112 or 168 bits (2-key / 3-key) | 32 to 448 bits (variable) |
| **Effective security** | 56 bits | About 80 (2-key) or 112 (3-key) bits | Up to 448 bits (key dependent) |
| **Structure** | 16-round Feistel | DES x 3 in EDE order (48 rounds) | 16-round Feistel with key-dependent S-boxes |
| **Brute-force resistance** | Very weak; broken in hours with dedicated hardware | Strong (2^112 effort for 3-key) | Strong with a long key |
| **Speed** | Fast | About 3x slower than DES (moderate) | Very fast after key setup |
| **Key setup cost** | Low | Low | High (slow key schedule) |
| **Standardisation** | Withdrawn (FIPS 46-3) | NIST SP 800-67; ANSI X9.52; ISO/IEC 18033-3 | No formal standard; public domain |
| **Banking adoption** | Obsolete | Widespread (ATM, EMV, PIN, HSMs) | Limited |
| **Suitability here** | Rejected | Selected | Alternative |

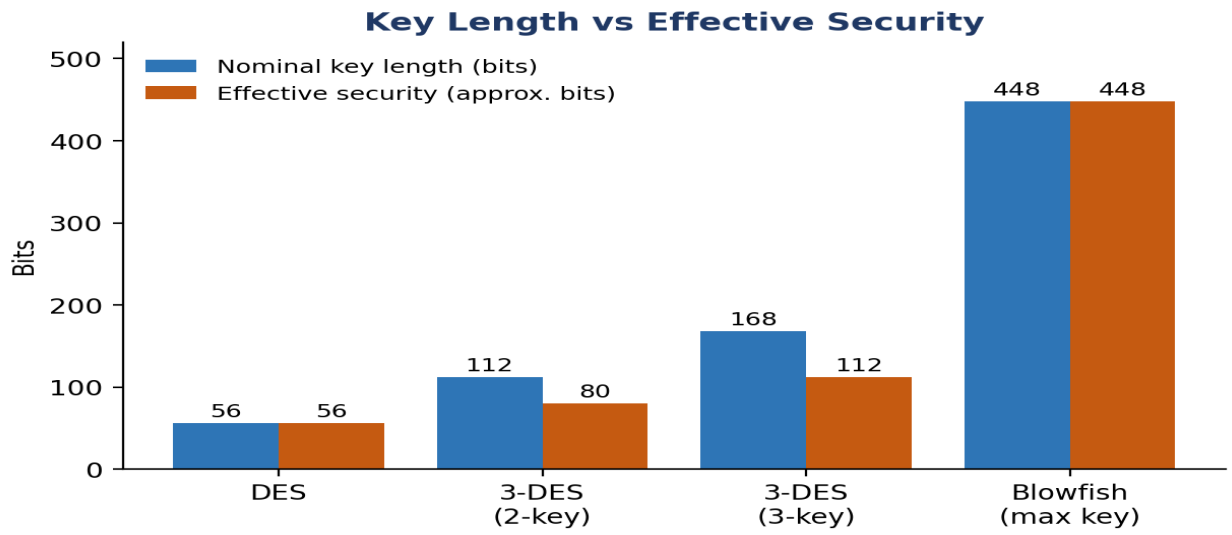



*Figure 2: Nominal key length versus effective security of the three algorithms*

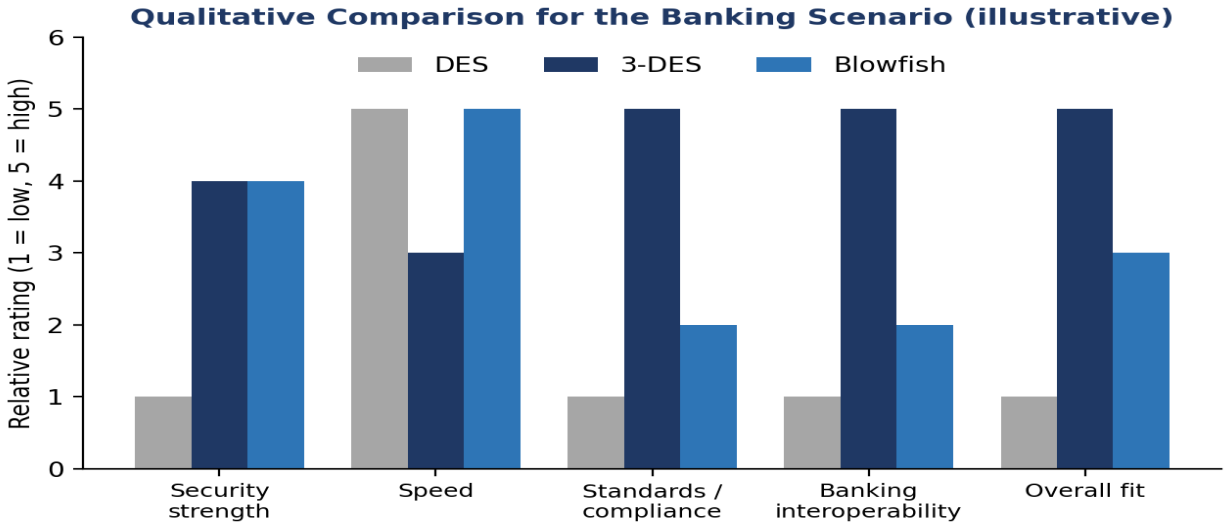



*Figure 3: Qualitative comparison against the banking requirements (illustrative ratings)*

## 4. Answer (a): Most Suitable Algorithm

> **Selected algorithm:** 3-DES (Triple DES, EDE mode with three independent keys) is the most suitable of the three options for the banking application.

## 5. Answer (b): Justification

### 5.1 Security Strength

- **DES is eliminated** because a 56-bit key cannot resist exhaustive search, so customer transactions would be exposed.
- **3-DES meets the 'high security' requirement.** Three-key EDE offers about 112-bit effective security, and the three-stage structure defeats the meet-in-the-middle attack that makes double DES only as strong as about 57 bits.
- **Blowfish is cryptographically strong** with long keys and has no practical attack on the full 16 rounds, but it has not been through formal standardisation, certification or the same degree of financial-industry review, which weakens the audit case.

### 5.2 Efficiency

- **The scenario requires moderate performance, not maximum speed.** Transactions are small messages, so 3-DES's roughly 3x cost over DES is acceptable on bank servers and hardware security modules.
- **Blowfish is faster,** but its advantage is not decisive here, and its slow key setup is inefficient when a fresh session key is used for each short transaction.
- **Hardware support:** 3-DES is implemented in payment HSMs and smart cards, so throughput can be scaled with dedicated hardware.

### 5.3 Standards, Compliance and Interoperability

3-DES is covered by NIST SP 800-67, ANSI X9.52 and ISO/IEC 18033-3, and is embedded in ATM networks, PIN processing and EMV chip cards. Choosing it allows the new application to interoperate with partner banks and payment networks without a protocol redesign, and satisfies audit expectations for a recognised, well-analysed cipher.

## 6. Proposed Secure Deployment

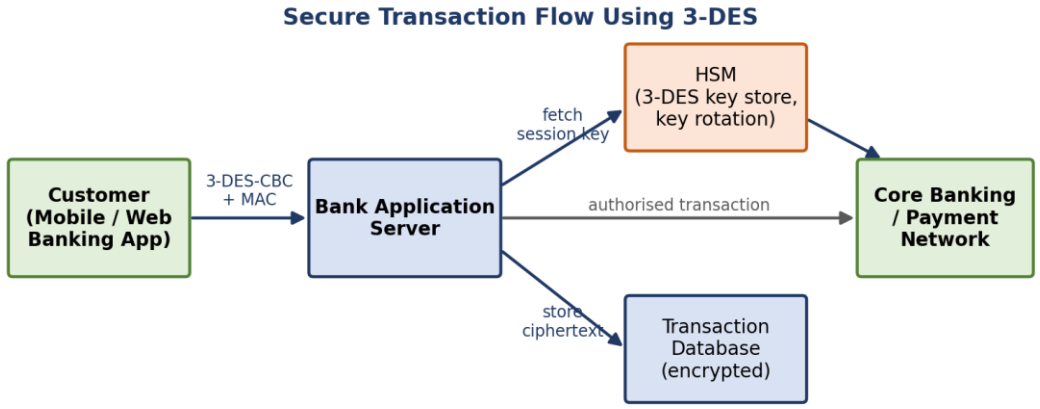



*Figure 4: Secure transaction flow using 3-DES in the banking application*

1. **Session key:** the application server obtains a fresh 3-DES session key (three independent 56-bit keys) from the HSM; keys never leave the HSM in clear form.
2. **Mode of operation:** each transaction is encrypted with 3-DES in CBC mode using a random, unpredictable IV; ECB is never used.
3. **Integrity:** a MAC (for example HMAC-SHA-256) is appended, since encryption alone does not detect tampering.
4. **Storage:** the encrypted transaction record is stored in the database, and keys are rotated on a defined schedule.
5. **Access control:** only authorised services can request keys; every key use is logged for audit.

## 7. Limitations and Mitigation

**Table 2: Known limitations of 3-DES and mitigations**

| Limitation | Risk | Mitigation |
|---|---|---|
| **64-bit block size** | Birthday-bound attacks such as Sweet32 after roughly 2^32 blocks (about 32 GB) under one key | Rotate keys frequently; cap data per key; use short-lived session keys |
| **Slower than modern ciphers** | Higher CPU load at scale | Use HSM acceleration and hardware offload |
| **Deprecated by NIST** | 3-DES is being phased out for new designs | Plan migration to AES-128/256 (AES-GCM) with crypto-agile design |
| **Weak modes or IV reuse** | Pattern leakage, replay | CBC with random IV plus MAC, never ECB |

## 8. Conclusion

Among the three options, **3-DES** is selected. It provides strong effective security (about 112 bits) that DES cannot, offers moderate performance that suits transaction-sized messages, and carries the standardisation and banking-industry support that Blowfish lacks. With key rotation, CBC mode with a MAC, and HSM-based key management, it meets the scenario's requirements. For long-term deployment, the bank should plan an upgrade path to AES.

# Scenario 5: Cryptography for IoT Devices

> **Scenario:** A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC). (a) Analyze the constraints of IoT devices. (b) Recommend a suitable cryptographic approach and justify the answer.

## 1. Understanding the Scenario

A smart home contains many small devices (door locks, sensors, lights, cameras, thermostats) that talk to a gateway and the cloud over wireless links such as BLE, Zigbee, Thread or Wi-Fi. They carry sensitive data (occupancy, video, lock commands), so each device must authenticate and agree on keys with its peers. Public key cryptography is needed for this key establishment and authentication, because it avoids pre-sharing a secret between every pair of devices. The question is which public key system the constrained devices can afford.

## 2. Answer (a): Constraints of IoT Devices

**Table 3: Constraints of IoT devices and their implications**

| Constraint | Typical limit | Implication for cryptography |
|---|---|---|
| **Processing power** | 8/16/32-bit MCU at a few MHz to ~100 MHz | Heavy big-integer arithmetic is slow; operations must be lightweight |
| **Memory** | A few KB to tens of KB RAM; 32 to 512 KB flash | Small keys, small code size, little working memory |
| **Battery / energy** | Coin cell or harvesting; years of life | Every CPU cycle and radio transmission drains the battery |
| **Bandwidth** | Packets of ~20 to 127 bytes (BLE, 802.15.4, LoRa) | Large keys, certificates and signatures need fragmentation and retransmission |
| **Latency** | Real-time control (locks, alarms) | Handshakes must complete quickly |
| **Physical exposure** | Devices installed in open areas | Risk of tampering and side-channel attacks; need secure key storage |
| **Scale and lifetime** | Hundreds of devices; 10+ years in service | Lightweight provisioning, updates and crypto-agility |

These constraints mean the chosen algorithm must offer strong security with small keys, low computation, low energy use and compact messages. This eliminates any approach that depends on large keys and expensive modular exponentiation.

## 3. Overview of RSA and ECC

### 3.1 RSA

RSA's security rests on the difficulty of factoring the product of two large primes. Encryption computes c = m^e mod n and decryption m = c^d mod n. Because known factoring algorithms (the general number field sieve) run in sub-exponential time, RSA needs very large moduli: 2048 bits for 112-bit security and 3072 bits for 128-bit security. Private-key operations (signing, decryption) require full-size modular exponentiation, and key generation requires finding large primes, both expensive on a microcontroller.

### 3.2 Elliptic Curve Cryptography

ECC's security rests on the elliptic curve discrete logarithm problem (ECDLP): given points G and Q = k·G on a curve, finding the scalar k is infeasible. The best known attacks are fully exponential, so ECC achieves the same security with much smaller keys. A 256-bit curve such as NIST P-256 or Curve25519 gives about 128-bit security. Typical uses are ECDH for key exchange and ECDSA or EdDSA for signatures.

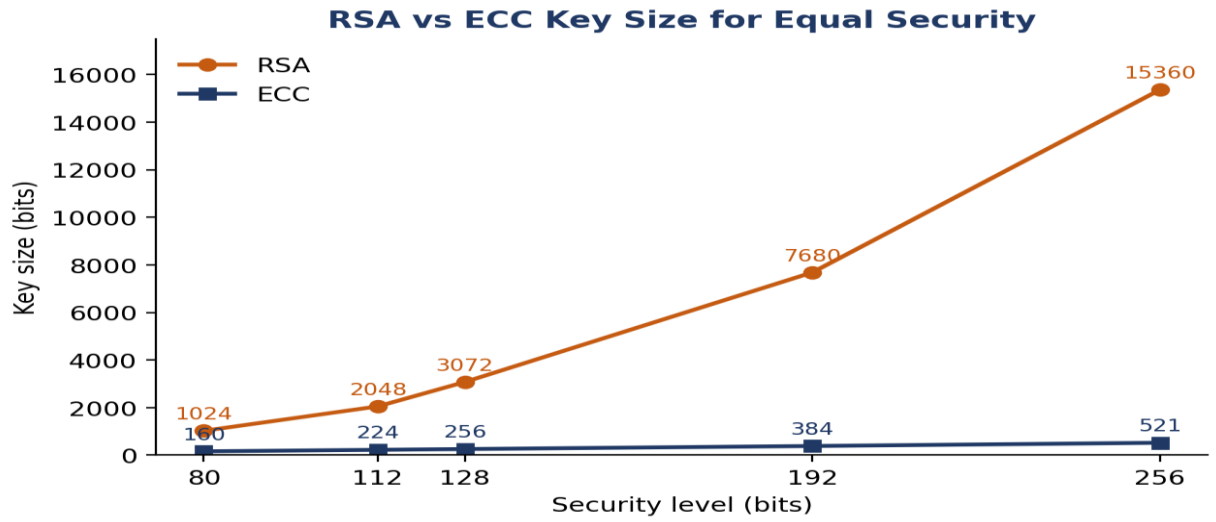



*Figure 5: Key size required by RSA and ECC for equal security levels*

**Table 4: Equivalent key sizes (NIST SP 800-57)**

| Security level (bits) | RSA key (bits) | ECC key (bits) | Approx. size ratio |
|---|---|---|---|
| **80** | 1024 | 160 | 6 : 1 |
| **112** | 2048 | 224 | 9 : 1 |
| **128** | 3072 | 256 | 12 : 1 |
| **192** | 7680 | 384 | 20 : 1 |
| **256** | 15360 | 521 | 30 : 1 |

## 4. RSA versus ECC on IoT Criteria

**Table 5: RSA versus ECC for constrained devices**

| Criterion | RSA | ECC |
|---|---|---|
| **Key size (128-bit security)** | 3072 bits | 256 bits |
| **Signature size** | 256 to 384 bytes | 64 bytes (ECDSA P-256 or Ed25519) |
| **Private-key operation** | Slow (modular exponentiation) | Fast (scalar multiplication) |
| **Public-key operation** | Fast with small exponent (e = 65537) | Slower than RSA verify, still modest |
| **Key generation** | Slow (prime search) | Fast (random scalar) |
| **RAM / flash use** | High | Low |
| **Energy and bandwidth** | High | Low |
| **Certificate size** | Large | Compact |
| **Suitability for IoT** | Poor for general use | Excellent |

## 5. Answer (b): Recommended Approach

> **Recommendation:** Use ECC (ECDH with Curve25519 or P-256 for key agreement, and ECDSA or Ed25519 for authentication) to establish session keys, and protect the data itself with a lightweight authenticated symmetric cipher such as AES-128-GCM, AES-128-CCM or ChaCha20-Poly1305.

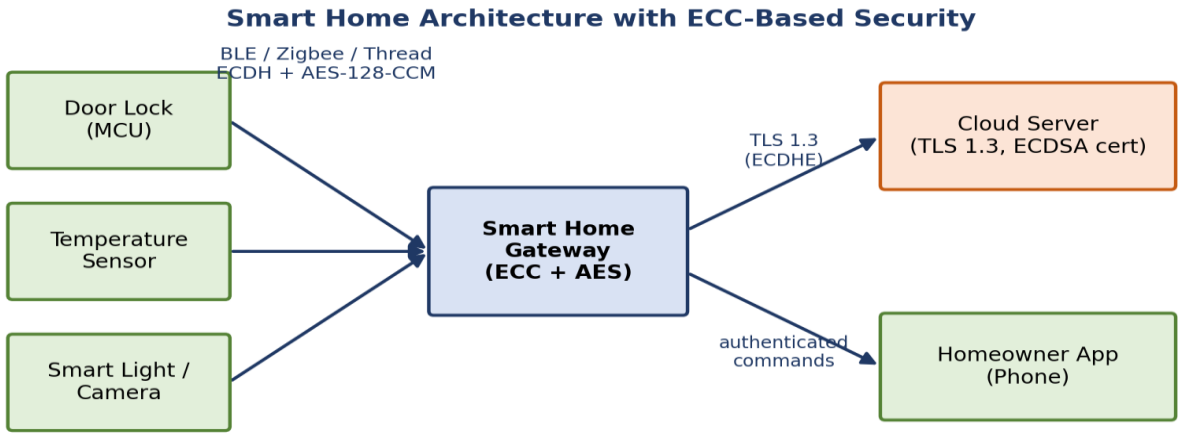



*Figure 6: Proposed smart home architecture with ECC-based security*

### 5.1 How the Secure Channel Works

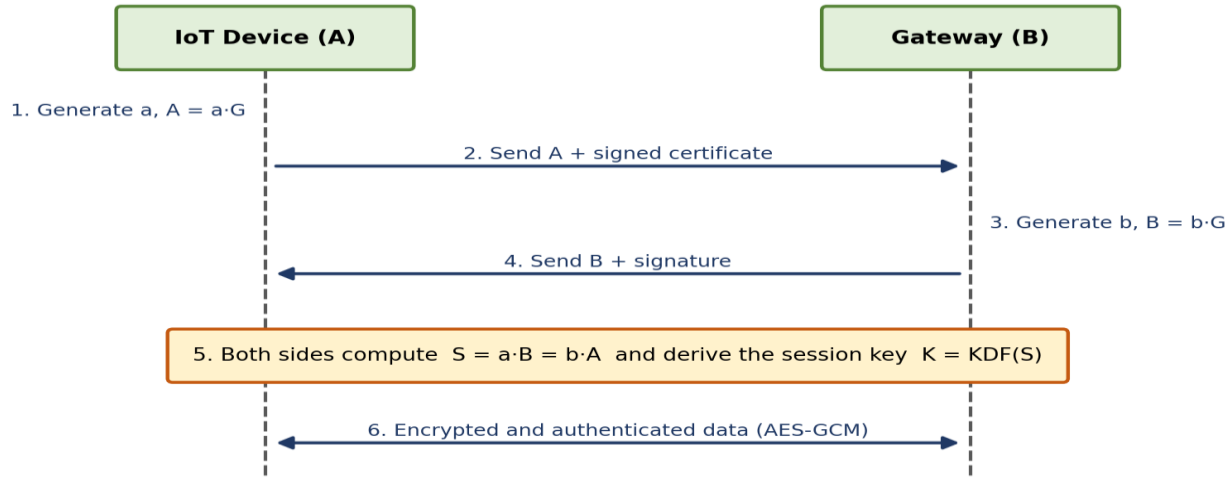



*Figure 7: ECDH key agreement and secure data exchange between an IoT device and the gateway*

1. **Key generation:** the device picks a random private scalar a and computes its public point A = a·G; the gateway does the same with b and B = b·G.
2. **Authentication:** each side sends its public point with an ECDSA or Ed25519 signature (or a compact certificate), so a man-in-the-middle cannot substitute keys.
3. **Shared secret:** both compute S = a·B = b·A without ever transmitting S.
4. **Key derivation:** a KDF such as HKDF turns S into a symmetric session key.
5. **Data protection:** all sensor readings and commands are encrypted and authenticated with AES-GCM or ChaCha20-Poly1305, which are fast and need little memory.

Public key operations are used only once per session, and the cheap symmetric cipher carries the traffic. This hybrid design keeps energy use low.

### 5.2 Why ECC Is the Better Choice

- **Small keys and signatures:** 32-byte keys and 64-byte signatures fit in a single BLE or 802.15.4 packet, while RSA-3072 would need several fragments.
- **Lower computation and energy:** faster private-key operations and key generation extend battery life and shorten handshakes, so locks and alarms respond promptly.
- **Less memory:** smaller keys, certificates and working buffers fit microcontrollers with only a few KB of RAM.
- **Strong security per bit:** a 256-bit curve matches RSA-3072, so there is no loss of security.
- **Ecosystem support:** ECC is supported by TLS 1.3, DTLS, Matter, Thread, mbed TLS, and hardware secure elements and crypto accelerators on common IoT chips.

## 6. Implementation Guidelines and Risks

**Table 6: Security risks and mitigations for ECC in IoT**

| Risk | Description | Mitigation |
|---|---|---|
| **Weak randomness** | Poor entropy exposes ECDSA private keys | Use a hardware TRNG; deterministic signatures (RFC 6979, Ed25519) |
| **Physical / side-channel attacks** | Devices can be opened or measured | Store keys in a secure element; constant-time implementations |
| **Man-in-the-middle** | Unauthenticated ECDH can be intercepted | Signed keys or certificates; device identity provisioned at manufacture |
| **Key compromise** | A single stolen key exposes a device | Per-device keys; ephemeral ECDH for forward secrecy; revocation |
| **Long device lifetime** | Algorithms may become obsolete (e.g. quantum computers) | Secure OTA firmware updates; crypto-agile design for post-quantum migration |

**Note:** RSA remains acceptable for one narrow case, a device that only verifies signed firmware images, because RSA verification with a small public exponent is cheap. It is not suitable as the main scheme for key exchange and signing on constrained devices.

## 7. Conclusion

ECC combined with symmetric authenticated encryption is the recommended approach for the smart home system. IoT devices are limited in processing power, memory, energy and bandwidth, and ECC delivers equivalent security to RSA with roughly one-twelfth of the key size at the 128-bit level, along with faster key generation and signing and compact messages. With a hardware random source, secure key storage, authenticated key exchange and a path to future algorithm upgrades, **ECC** provides secure, efficient communication for low-power IoT devices.

## References


1. W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
2. NIST SP 800-57 Part 1 Rev. 5, Recommendation for Key Management (key sizes and security strengths).
3. NIST SP 800-67 Rev. 2, Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher.
4. B. Schneier, Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish), 1994.
5. K. Bhargavan and G. Leurent, On the Practical (In-)Security of 64-bit Block Ciphers (Sweet32), ACM CCS 2016.
6. RFC 7748, Elliptic Curves for Security (Curve25519, Curve448).
7. RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3.

RFC 6979, Deterministic Usage of DSA

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Understanding of Scenario | 20% | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20% | Accurately identifies all relevant issues and factors involved | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues |
| Application of Concepts | 25% | Applies appropriate concepts effectively to address the scenario | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts |
| Decision Making | 20% | Makes well-reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided |
| Clarity of Response | 15% | Response is clear, structured, and logically presented | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand |